Informe Grupo 04

Desarrollo de un chatbot para la predicción del éxito en el ofrecimiento de una suscripción a un depósito a plazo

Asencion Anchiraico Jerson D.
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e-2025100767M@uncp.edu.pe

Hinostroza Surichaqui Jhordan
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e-2025100778k@uncp.edu.pe

Huallpa Apaza Jhon I.
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e-2025100780M@uncp.edu.pe

Lizano Remigio Luis C.
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e-2025100784D@uncp.edu.pe

***Abstract*—Este documento presenta el desarrollo e implementación de una solución de Machine Learning servida mediante una API REST y un asistente conversacional que indica a un asesor de ventas telefónicas si conviene llamar a un cliente para ofrecerle un depósito a plazo fijo. Utilizando el dataset UCI Bank Marketing de 45,211 registros, se construyó un pipeline de preprocesamiento sin fuga de información (*data leakage*) y se seleccionaron 5 de las 15 variables mediante un ranking de correlación. Se compararon un modelo trivial, una regresión logística y dos redes neuronales, eligiendo el umbral de decisión de cada modelo en validación según el costo de los errores. La red promovida (MLP V2) encuentra al 51% de los clientes que suscriben y reduce el costo en 35% frente al modelo trivial.**

***Index Terms*—Machine Learning, Bank Marketing, Depósitos a Plazo, Fuga de Información, Umbral por Costo, FastAPI, Chatbot.**

I. Introducción y Planteamiento del Problema

A. Introducción

En el sector bancario, las campañas de marketing directo por vía telefónica representan uno de los canales principales para la captación de capital mediante depósitos a plazo fijo [2]. Sin embargo, realizar llamadas masivas sin un filtro de prospección genera altos costos operativos y la insatisfacción de los clientes. El presente trabajo propone un sistema inteligente de soporte a la decisión comercial basado en modelos de aprendizaje automático y un asistente conversacional interactivo.

B. Definición del Problema y Usuario Final

El sistema indica, antes de entablar contacto directo, si conviene llamar a un cliente para ofrecerle un depósito a plazo fijo.

- **Usuario objetivo:** Ejecutivos comerciales y asesores de venta telefónica de la institución financiera.
- **Decisión que apoya:** A qué clientes llamar primero, para optimizar la tasa de conversión de la fuerza de ventas.
- **Respuesta del sistema: Sí** (conviene llamar) o **No** (no priorizar por ahora).

C. Momento de la Predicción y Prevención de Fuga de Información

La inferencia del modelo se realiza estrictamente **antes de efectuar la llamada telefónica**.

> **Tratamiento de Fuga de Información (*Data Leakage*):** La variable `duration` (duración de la llamada) fue eliminada explícitamente de las características de entrada (X). Como indica la propia documentación del dataset [1] y la literatura técnica [3], esta variable solo se conoce *después* de finalizar la llamada. Incluirla alteraría artificialmente el rendimiento del modelo sin validez operacional real. En cambio, el mes de la llamada sí se conoce de antemano, porque la llamada se programa.

D. Análisis Económico de Errores y Métrica Objetiva

Los errores del modelo conllevan costos asimétricos para la institución:

1) **Falso Positivo (FP):** Clasificar a un cliente no interesado como potencial suscriptor. Genera un costo operativo menor (una llamada de más). Se le asigna un costo de 1.
2) **Falso Negativo (FN):** Clasificar a un cliente interesado como no prospecto. Genera la pérdida de la venta (costo de oportunidad). Se le asigna un costo de 5.

Al ser el costo por FN mayor que el de FP, en un contexto de clases desbalanceadas [6], el umbral de decisión no es 0.5: se elige en validación como el que minimiza el costo total. La métrica de decisión es el **costo por cliente**, $(5 \cdot FN + 1 \cdot FP)/n$, complementada con Precision, Recall, F_1-Score y el área bajo la curva Precision-Recall (**PR-AUC**), más informativa que ROC-AUC cuando la clase positiva es minoritaria [5].

E. Riesgo Ético y Mitigación

El dataset incluye atributos sensibles (`age`, `marital`, `job` y `education`). Para reducir el riesgo de discriminación, el modelo usa solo uno de ellos (`job`), se mide si el error cambia entre ocupaciones y el sistema siempre permite la derivación a un asesor humano. El sistema solo prioriza llamadas: no niega productos.

II. Datos y Exploración

A. Limpieza y Preprocesamiento

Sobre el dataset UCI Bank Marketing (45,211 registros, sin valores nulos) [1], [2], se detectaron y resolvieron los problemas de calidad resumidos en la Tabla I:

TABLE I
PROBLEMAS DE CALIDAD Y DECISIONES DE LIMPIEZA

Problema	Decisión	Justificación
`duration`	Eliminada	Se conoce después de la llamada (fuga)
16 duplicados al quitar `duration`	Eliminados antes de particionar	Evitar que un cliente esté en entrenamiento y prueba
`"unknown"` (82% en `poutcome`)	Categoría propia	Son muchos y sí informan
Desbalance 88/12	Estratificar y pesos de clase	La exactitud engaña

Las variables categóricas se transforman mediante *One-Hot Encoding* con Scikit-Learn [4]; las categorías no vistas durante el entrenamiento se ignoran sin generar error.

B. Particionado Estratificado sin Fuga

Tras la limpieza (45,195 registros), se dividió el conjunto de datos con la semilla `random_state=42` en tres subconjuntos independientes, manteniendo la proporción de clases (11.7% positivos en cada uno):

- **Entrenamiento (70%):** 31,638 registros.
- **Validación (15%):** 6,777 registros.
- **Prueba (15%):** 6,780 registros.

Las particiones se guardan sin escalar. El preprocesador realiza el ajuste (`fit`) únicamente sobre el conjunto de **entrenamiento** y se guarda en `modelos/preprocesador.pkl`, para que la API aplique exactamente la misma transformación.

C. Selección de Variables mediante Correlación

Se ordenaron las 15 variables según su asociación con la suscripción, calculada solo con entrenamiento:

- **Numéricas:** valor absoluto de la correlación de Pearson con y.
- **Categóricas:** V de Cramér [11], que mide la asociación entre categorías sin asumir un orden.

La matriz de correlación con categorías codificadas como 0, 1, 2... (`OrdinalEncoder`) introduce un orden inexistente (los meses quedan en orden alfabético): con ella, `month` queda en el puesto 12, cuando su tasa de suscripción va de 6.6% (mayo) a 50.6% (marzo). Con V de Cramér pasa al puesto 2 (Tabla II y Fig. 1).

TABLE II
RANKING DE ASOCIACIÓN CON LA SUSCRIPCIÓN (ENTRENAMIENTO)

Puesto	Variable	Medida	Valor	Con OrdinalEnc.
1	**poutcome**	Cramér	0.318	5
2	**month**	Cramér	0.258	12
3	**contact**	Cramér	0.151	1
4	**housing**	Cramér	0.143	2
5	**job**	Cramér	0.137	11
6	pdays	Pearson	0.101	3
7	previous	Pearson	0.093	4

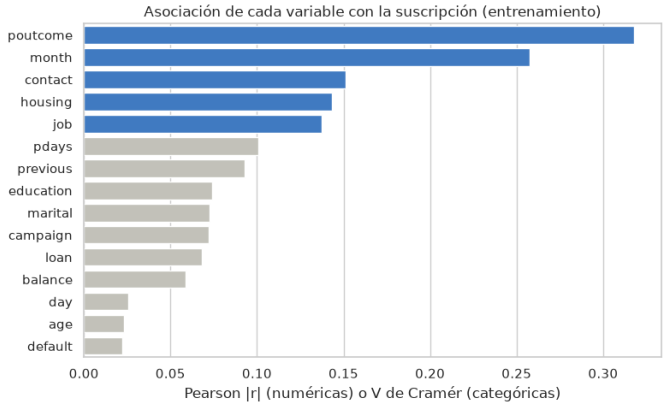


Fig. 1. Asociación de cada variable con la suscripción. En azul, las 5 variables elegidas.

Se eligieron las **5 variables con mayor asociación**: **`poutcome`, `month`, `contact`, `housing` y `job`**. Después del quinto puesto la asociación disminuye de forma apreciable. Así, el chatbot pide exactamente los 5 datos que usa el modelo, sin completar ninguno con valores supuestos, y se usa una sola variable demográfica en lugar de cuatro. Tras el *One-Hot Encoding*, estas 5 variables producen 33 columnas de entrada.

III. Etapa 3: Modelado

A. Enfoques de Modelado Implementados

Con el fin de resolver el problema de clasificación binaria, se diseñó un flujo de modelado en Python con Scikit-Learn [4] y Keras [10]. Se evaluaron cuatro enfoques de complejidad progresiva, todos con las 5 variables elegidas:

1) **Modelo Trivial (Dummy):** Predice siempre la clase mayoritaria ("no suscribe"), permitiendo dimensionar el sesgo del dataset desbalanceado.
2) **Baseline Simple (Regresión Logística):** Modelo lineal con pesos de clase balanceados (`class_weight="balanced"`).
3) **Red Neuronal MLP V1:** Perceptrón multicapa con una capa oculta de 32 neuronas (ReLU).
4) **Red Neuronal MLP V2:** Arquitectura más profunda (64–32) con regularización *Dropout* (0.3 y 0.2) [9].

Ambas redes usan el optimizador Adam [8], un máximo de 80 épocas con *batch size* de 64, pesos de clase (un error con

un cliente que sí suscribe pesa unas 7.5 veces más) y parada temprana (*Early Stopping*) con paciencia de 5 épocas sobre la pérdida de validación [7]. Se fija la semilla 42 en TensorFlow para que el entrenamiento sea reproducible. La MLP V1 se detuvo en la época 27 y la MLP V2 en la 15.

B. Análisis de Resultados y Rendimiento

Los modelos se entrenaron con 31,638 registros y se evaluaron en validación (6,777 registros), cada uno con su umbral elegido por costo. Los resultados se consolidan en la Tabla III.

TABLE III
RENDIMIENTO EN VALIDACIÓN (UMBRAL ELEGIDO POR COSTO)

Modelo	Umbral	Acc	Prec.	Recall	F_1	Costo
1. Trivial (Dummy)	–	0.883	0.000	0.000	0.000	0.585
2. Lineal Simple (LogReg)	0.57	0.852	0.396	0.498	0.441	0.383
3. Red Neuronal V1 (MLP)	0.62	0.858	0.414	0.515	0.459	0.369
4. **Red Neuronal V2 (MLP+Drop)**	0.58	0.854	0.405	0.525	0.457	**0.3685**

C. Discusión Técnica

Los hallazgos de la Etapa 3 demuestran que:

- **Limitación del Accuracy:** El modelo trivial alcanza 88.3% de exactitud sin encontrar a ningún suscriptor (Recall 0), evidenciando la invalidez de usar la exactitud por sí sola [6]. Su PR-AUC es 0.117, igual a la proporción de positivos.
- **Red frente a línea base:** Las dos redes reducen el costo frente a la regresión logística (0.369 frente a 0.383), pero la mejora es pequeña; la regresión logística sigue siendo una alternativa razonable y más interpretable.
- **Complejidad:** La MLP V2 y la MLP V1 quedan prácticamente empatadas (0.3685 frente a 0.3693). Con solo 5 variables categóricas, una red más profunda no aporta una mejora clara.

IV. Etapa 4: Evaluación y Versionado

A. Elección del Umbral y del Modelo

El modelo y el umbral se eligieron **solo con validación**. Para cada modelo se probaron los umbrales de 0.01 a 0.99 y se eligió el de menor costo total. El criterio para elegir la versión ganadora es el **menor costo por cliente en validación**: resultó la MLP V2, con umbral **0.58** (Fig. 2).

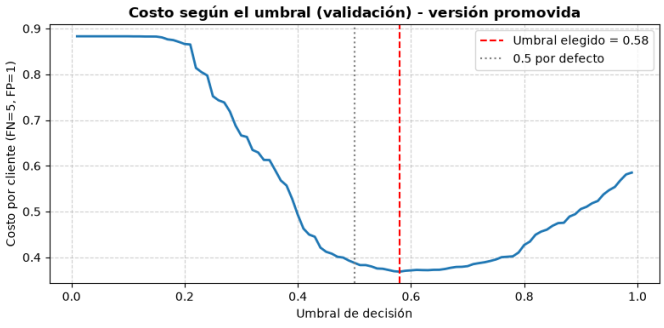


Fig. 2. Costo por cliente según el umbral (validación, versión promovida).

B. Evaluación Final en Prueba

La evaluación en prueba se realizó **una sola vez**, con la versión ya promovida (Tabla IV y Fig. 3). El modelo encuentra 406 de los 793 clientes que suscribieron, con 628 llamadas que no resultan, y reduce el costo en 35% frente al modelo trivial. Las métricas de prueba son similares a las de validación.

TABLE IV
MLP V2 EN PRUEBA (UMBRAL 0.58)

Acc	Prec.	Recall	F_1	PR-AUC	Costo
0.850	0.393	0.512	0.444	0.386	0.378

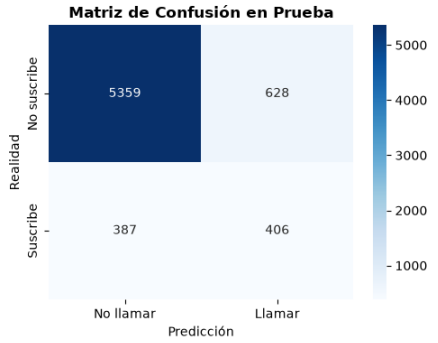


Fig. 3. Matriz de confusión en prueba (MLP V2, umbral 0.58).

C. Versionado y Trazabilidad

Cada versión tiene su propia ficha técnica JSON (`ficha_trivial.json`, `ficha_logreg.json`, `ficha_mlp_v1.json` y `ficha_mlp_v2.json` en `modelos/`) con la fecha de entrenamiento, los datos (archivo, filas, semilla y tamaño de cada conjunto), las variables, los hiperparámetros, el umbral y su criterio y las métricas de validación. Solo la versión promovida tiene métricas de prueba y el análisis por ocupación. El archivo `modelos/modelo_promovido.json` indica la versión que usa la API, su umbral y el motivo de su elección (Anexo 1).

Como los pesos de clase inflan el puntaje del modelo, este no se presenta como una probabilidad: la API y el chatbot responden únicamente **sí** o **no**.

V. Etapa 5: API de Inferencia (Despliegue del Modelo)

A. Arquitectura y Diseño del Servicio Web

El despliegue del modelo promovido se implementó con **FastAPI** [12] (`src/api.py`), por su validación estricta de esquemas y su documentación interactiva automática en Swagger UI (`/docs`). Sus componentes clave son:

- **Carga única del modelo:** Al iniciar, la API lee `modelo_promovido.json` y carga en memoria RAM el modelo ganador, su preprocesador y su umbral. Promover otra versión solo requiere reiniciar la API.

- **Endpoint de Salud (`GET /salud`):** Retorna el estado del servicio, si el modelo está cargado, la versión y las 5 variables que espera.
- **Endpoint de Predicción (`POST /predecir`):** Recibe las 5 variables y devuelve la respuesta ("sí"/"no"), el puntaje del modelo sin modificaciones, el umbral y la versión. La respuesta es "sí" cuando el puntaje es mayor o igual que el umbral.
- **Validación de Entradas con Pydantic:** El esquema `ClienteEntrada` (`src/preprocesamiento_api.py`) usa enumeraciones (`Enum`) con los valores del dataset y rechaza campos adicionales como `duration`. Un campo faltante, un valor fuera de las opciones o un campo no permitido devuelve el **código HTTP 400** con mensajes en español. Si no hay modelo cargado, devuelve 503.
- **Middleware CORS:** Permite la comunicación con la interfaz web local (`index.html`).

B. Evidencia de Funcionamiento

El servicio se ejecuta localmente mediante el comando:

```
uvicorn src.api:app --reload --port 8000
```

Ejemplo de solicitud y respuesta:

```
{"poutcome": "success", "month": "oct",
 "contact": "cellular", "housing": "no",
 "job": "management"}
-> {"respuesta": "sí", "predice_suscripcion": true,
    "etiqueta": "yes", "puntaje": 0.9743,
    "umbral": 0.58,
    "version_modelo": "MLP_Version_2"}
```

VI. Etapa 6: Chatbot Asistente Conversacional

A. Diseño del Flujo Conversacional y Arquitectura

Se implementó un asistente conversacional en `src/chatbot.py`, que recolecta los datos paso a paso y consume la API desarrollada en la sección anterior. El flujo contempla tres caminos (diagrama en el Anexo 2):

1) **Camino Feliz (Happy Path):** El usuario responde las 5 preguntas, el chatbot construye el JSON con esos datos, consulta la API y muestra el resultado.
2) **Camino Alternativo:** Si el usuario escribe una respuesta que no corresponde a ninguna opción, el chatbot avisa "intento N de 3", muestra las opciones válidas y repite la pregunta.
3) **Camino de Error / Fallback y Derivación a Humano:** Al tercer intento fallido, si el usuario lo pide (*"humano"*, *"asesor"*, *"persona"*) o si la API no responde o devuelve un error, el chatbot deriva la atención a un asesor comercial.

B. Recolección de Datos Paso a Paso (Slots)

El chatbot solicita exactamente las 5 variables del modelo y traduce respuestas en español a los valores del dataset:

- Resultado de la campaña anterior (`poutcome`: éxito, fallo, desconocido, otro).
- Mes de la llamada (`month`: nombre del mes o su número).
- Medio de contacto (`contact`: celular, teléfono, desconocido).
- Crédito hipotecario activo (`housing`: sí, no).
- Ocupación del cliente (`job`: administrativo, obrero, gerente, jubilado, estudiante, etc.).

No se completan datos con valores supuestos ni se envía `duration`.

C. Interpretación No Técnica y Mensajes Comprensibles

El chatbot traduce la respuesta de la API a un lenguaje comercial claro: **SÍ, conviene llamar a este cliente** o **NO, no priorizar por ahora**, con una sugerencia (incluirlo en la lista de llamadas o atender primero a otros clientes). No muestra porcentajes, porque el puntaje no es una probabilidad real. El chatbot no solicita nombre ni DNI.

Para ejecutar el asistente en una terminal independiente (con la API encendida), se utiliza:

```
python -m src.chatbot
```

VII. Limitaciones y Ética

El despliegue de modelos predictivos en el sector bancario exige una revisión rigurosa de sus limitaciones técnicas y de las implicaciones éticas asociadas a las decisiones automatizadas.

A. Limitaciones Técnicas y Metodológicas

- **Vigencia de los Datos (*Concept Drift*):** Los datos corresponden a un banco portugués entre 2008 y 2010, y la partición es aleatoria, no temporal. El efecto del mes o de las campañas anteriores podría no repetirse hoy, por lo que el modelo debe reentrenarse periódicamente.
- **Costos Supuestos:** Los costos $FN = 5$ y $FP = 1$ son un supuesto que el banco debería validar; si cambian, cambia el umbral.
- **Mejora Moderada y Variabilidad:** La red supera a la regresión logística por un margen pequeño, y la diferencia entre la MLP V1 y la MLP V2 es mínima. No se midió la variabilidad con distintas semillas, por lo que esa diferencia podría invertirse.
- **Selección de Variables:** Las 5 variables se eligieron con el ranking de correlación; no se comparó en validación el rendimiento con más variables.

B. Consideraciones Éticas y Responsabilidad

- **Prevención de Sesgos Algorítmicos:** De las cuatro variables demográficas, el modelo solo usa la ocupación. Se midió el error en prueba por ocupación: el modelo encuentra más suscriptores entre estudiantes (Recall 0.85) y jubilados (0.78) y menos entre obreros (0.28). Esta tabla queda en la ficha técnica para revisarla en cada nueva versión.
- **Privacidad:** El chatbot no pide datos personales identificables ni guarda las respuestas.

- **Supervisión Humana y Acoso Comercial:** El sistema solo prioriza llamadas, no niega productos, siempre permite derivar a un asesor humano y busca reducir el volumen total de llamadas innecesarias.

VIII. Contribución de cada integrante

El desarrollo y la ejecución técnica del proyecto *Bank Marketing* (Grupo 04) se llevaron a cabo de manera colaborativa y estructurada, asignando responsabilidades específicas a cada miembro del equipo:

- **Luis Lizano (25%):** Exploración analítica de datos en la Etapa 2 (EDA, identificación de patrones y aislamiento de la variable `duration`), y ejecución de la Etapa 4 de Evaluación y Versionado (selección del modelo en validación, prueba final y generación de las fichas técnicas JSON).
- **Jerson Asencion (25%):** Configuración e inicialización del entorno de desarrollo virtual (`.venv`), arquitectura modular del proyecto y diseño de la API para la integración del modelo predictivo.
- **Jhon Huallpa (25%):** Construcción y entrenamiento de arquitecturas en la Etapa 3 de Modelado, diseño del diagrama de flujo del proceso y desarrollo del módulo del chatbot interactivo.
- **Jhordan Hinostroza (25%):** Creación, estructuración e integración de los *notebooks* analíticos correspondientes a las Etapas 2, 3 y 4 del *pipeline* del proyecto.

IX. Declaración de uso de herramientas de IA

En cumplimiento de las políticas de integridad académica y transparencia en el desarrollo del proyecto, el equipo del **Grupo 04** declara el uso asistido de herramientas de Inteligencia Artificial Generativa durante el ciclo de vida del trabajo. A continuación, se detallan las herramientas utilizadas, sus aplicaciones específicas y los procesos de verificación manual realizados por los integrantes.

A. Herramientas Utilizadas y Propósito

- **Google Gemini / ChatGPT (Modelos LLM):** Utilizados como asistentes para la estructuración conceptual del informe, revisión semántica y gramatical del texto en español, apoyo en la sintaxis de plantillas LaTeX (IEEE) y generación de código base de prueba para los scripts del *pipeline* en Python (`pandas`, `scikit-learn`, `tensorflow/keras`).
- **GitHub Copilot:** Utilizado como autocompletado de código dentro del entorno de desarrollo (VS Code) para acelerar la escritura de funciones repetitivas en los módulos de datos y evaluación.
- **Claude (Anthropic):** Utilizado para revisar el repositorio y proponer correcciones puntuales (eliminación de duplicados, selección de variables por correlación, umbral por costo, API, chatbot y pruebas automáticas).

B. Proceso de Verificación y Validación Manual

Ningún resultado o código generado por las herramientas de IA fue incorporado al proyecto sin previa revisión, ajuste y validación directa por parte de los miembros del grupo:

- **Verificación del Negocio y Lógica de Datos:** La decisión crítica de excluir la variable `duration` para evitar la fuga de datos (*Data Leakage*), así como la definición de los costos de los errores, fue conceptualizada, debatida y validada manualmente por el equipo antes del entrenamiento.
- **Auditoría y Ejecución del Código:** Se revisaron los módulos `src/datos.py`, `src/modelo.py` y `src/evaluacion.py`, y se ejecutaron `python entrenar.py` y las 19 pruebas automáticas (`pytest`) dentro del entorno virtual (`.venv`) para garantizar que no existieran errores de ejecución.
- **Validación de Resultados a Nivel de Métricas:** Todos los valores reportados (umbral, Precision, Recall, PR-AUC, matriz de confusión) provienen de ejecuciones reales registradas en las fichas técnicas de `modelos/`.
- **Revisión Editorial del Informe:** La redacción final del documento fue revisada críticamente para asegurar el tono académico, el rigor técnico y la coincidencia con los entregables requeridos.

References

[1] S. Moro, P. Rita, and P. Cortez, "Bank Marketing," UCI Machine Learning Repository, 2014. [Online]. Available: https://doi.org/10.24432/C5K306

[2] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," *Decision Support Systems*, vol. 62, pp. 22–31, 2014.

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015.

[6] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009.

[7] L. Prechelt, "Early stopping — but when?," in *Neural Networks: Tricks of the Trade*. Berlin: Springer, 1998, pp. 55–69.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.

[10] F. Chollet *et al.*, "Keras," 2015. [Online]. Available: https://keras.io

[11] H. Cramér, *Mathematical Methods of Statistics*. Princeton, NJ: Princeton University Press, 1946.

[12] S. Ramírez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com

X. Anexos

A. Anexo 1: Ficha técnica

Extracto de `modelos/modelo_promovido.json` y de la ficha técnica de la versión promovida:

```
modelo_promovido.json
  modelo:  MLP_Version_2
  archivo: modelos/mlp_v2.keras
  umbral:  0.58
```

```
  motivo:  Menor costo por cliente en
           validacion (0.3685).

modelos/ficha_mlp_v2.json
  datos:            bank-full.csv, 45195 filas,
                    semilla 42,
                    31638 / 6777 / 6780
  variables:        poutcome, month, contact,
                    housing, job
  hiperparametros:  capas [64, 32],
                    dropout [0.3, 0.2], adam,
                    epocas_max 80, batch 64,
                    patience 5, class_weight
  umbral:           0.58 (minimo costo,
                    FN = 5, FP = 1)
  metricas_prueba:  precision 0.3926,
                    recall 0.512, pr_auc 0.3862,
                    costo 0.378
```

B. Anexo 2: Diagrama de flujo del chatbot

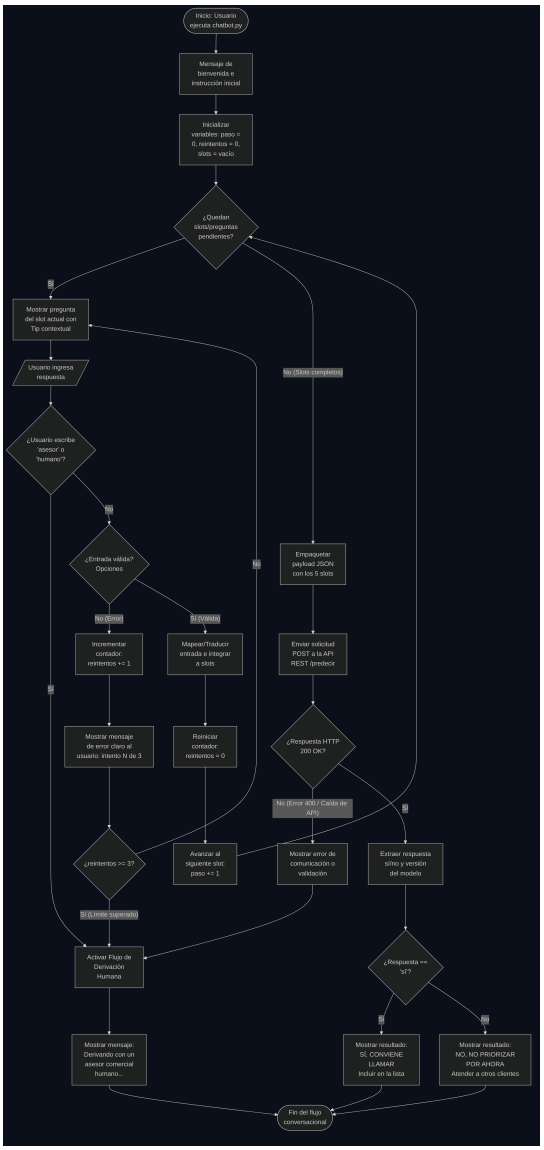


Fig. 4. Diagrama de flujo del chatbot (caminos feliz, alternativo y de error)

C. Anexo 3: Evidencia de operación



Fig. 5. Operación de la API (`/docs`): respuesta "sí" y error 400

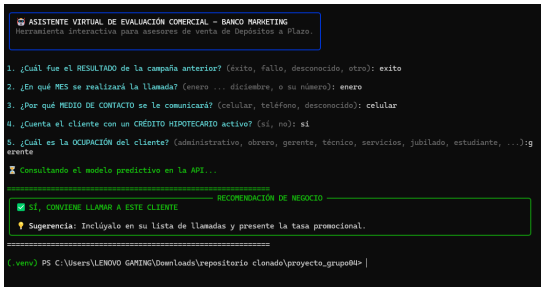


Fig. 6. Operación del Chatbot en consola